

UE24CS252B - Computer Networks
Socket Programming Mini Project

Title: Secure Publisher-Subscriber Message Broker

Submitted By:

Manasi Dinesh Unnithan	PES2UG24CS260
Mohith Kumar Battu	PES2UG24CS283
N A Manasa	PES2UG26CS291

Abstract

This project implements a Publisher-Subscriber message broker using socket programming and TLS encryption for secure communication. The system enables multiple publishers to send messages to subscribed clients through a centralized broker server. The broker handles concurrent client connections using multi-threading, with synchronization mechanisms to protect shared data structures. SSL/TLS integration ensures encrypted communication channels between clients and the broker. A built-in performance monitor tracks message throughput and active client count in real time.

System Overview

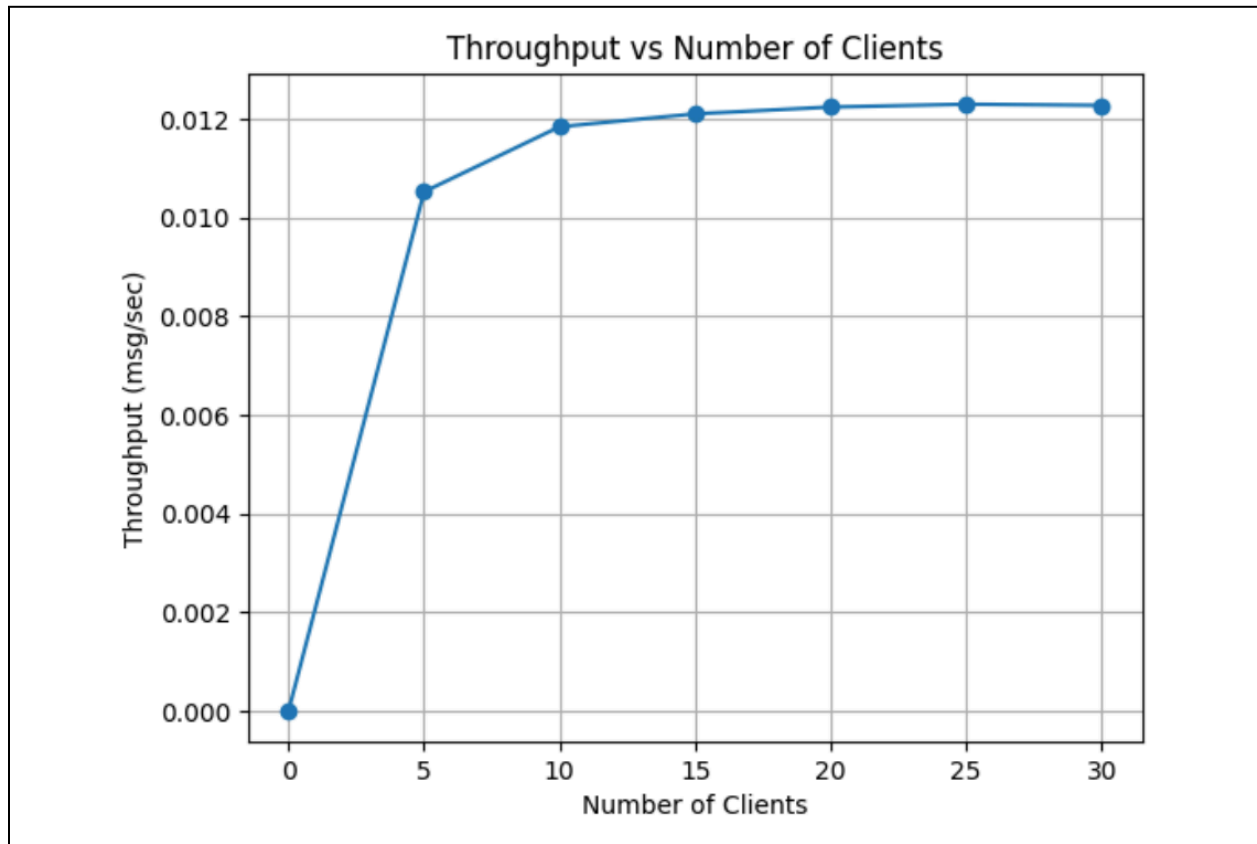
The **Broker** server runs on port 5000 and manages all communication between publishers and subscribers. It stores topic information and maintains a list of connected clients, both protected by a mutual exclusion lock to prevent conflicts during simultaneous access. When a client connects, the broker creates a new thread to handle that client's requests.

Publishers connect to the broker and send messages in the format: **PUBLISH:topic:message**. The broker then forwards these messages to all subscribers listening to that topic. Publishers receive an acknowledgment when the message is successfully delivered.

Subscribers connect and request messages from specific topics using the command: **SUBSCRIBE:topic**. Once subscribed, a background thread continuously listens for incoming messages and displays them to the user. This allows subscribers to receive messages without blocking other operations.

Security is provided through TLS/SSL encryption. The broker loads a certificate and private key at startup. Clients establish encrypted connections and disable hostname checking to work with the self-signed certificate. If the SSL handshake fails, the broker logs the error and closes the connection without crashing.

Graph Analysis: Throughput vs. Number of Clients



Note: For the purpose of maintaining only one dependant variable, a constant number of messages at 10messages per 5 clients added was maintained

The experimental graph shows three distinct performance phases:

Phase 1 (0–10 clients): *Rapid Growth*; Throughput increases sharply as clients are added. The system has plenty of available resources, and thread creation overhead is minimal. Each new client contributes significantly to overall performance.

Phase 2 (10–20 clients): *Saturation*; Growth slows noticeably. As client count increases, the broker begins managing many threads simultaneously, leading to increased overhead from context switching. The single mutual exclusion lock protecting shared data becomes a bottleneck when multiple publishers access topics simultaneously.

Phase 3 (20+ clients): *Plateau*; Throughput stabilizes at approximately 0.012 messages per second. Additional clients no longer improve performance because system resources are fully utilized. The overhead of thread management, synchronization, and encrypted communication now exceeds any benefit from additional clients.

Output Screenshots:

Broker:

Broker connected and waiting for clients
(Publisher and Subscribers to connect)

```
Secure Broker Server running...
[CONNECTED] ('127.0.0.1', 48405)

===== PERFORMANCE =====
Active clients: 1
Topics: 0
Messages processed: 0
Throughput: 0.0 msg/sec
=====

[CONNECTED] ('127.0.0.1', 48407)

===== PERFORMANCE =====
Active clients: 2
Topics: 0
Messages processed: 0
Throughput: 0.0 msg/sec
=====
```

1 topic established and message sent within the topic:

```
===== PERFORMANCE =====
Active clients: 2
Topics: 1
Messages processed: 0
Throughput: 0.0 msg/sec
=====

===== PERFORMANCE =====
Active clients: 2
Topics: 1
Messages processed: 1
Throughput: 0.01 msg/sec
=====

===== PERFORMANCE =====
Active clients: 2
Topics: 1
Messages processed: 1
Throughput: 0.01 msg/sec
=====
```

Number of Clients, Messages and Topics Increased:

```
===== PERFORMANCE =====  
Active clients: 4  
Topics: 2  
Messages processed: 2  
Throughput: 0.01 msg/sec  
=====
```

```
===== PERFORMANCE =====  
Active clients: 4  
Topics: 2  
Messages processed: 2  
Throughput: 0.01 msg/sec  
=====
```

```
===== PERFORMANCE =====  
Active clients: 4  
Topics: 2  
Messages processed: 3  
Throughput: 0.01 msg/sec  
=====
```

```
===== PERFORMANCE =====  
Active clients: 4  
Topics: 2  
Messages processed: 3  
Throughput: 0.01 msg/sec  
=====
```

Publisher:

Publisher1 - established topic News and sends message

```
PS C:\Users\manas\Sem4\cn\pubsub_service\publisher> python publisher_client.py  
Secure publisher connected  
Topic: News  
Message: Iran Seizes Gujrati Ships passing through the Strait of Hormuz  
Server: OK
```

Publisher2 - established topic Sports and sends message

```
Secure publisher connected
Topic: Sports
Message: RR won against LSG by 40 runs
Server: OK

Topic: Sports
Message: next match: CSK vs MI
Server: OK
```

Subscriber:

Subscriber1- subscribes and receives broadcasted messages from topic News

```
PS C:\Users\manas\Sem4\cn\pubsub_service\subscriber> python subscriber_client.py
Secure subscriber connected
Topic to subscribe: News

Received: SUBSCRIBED:News

Received: MESSAGE:News:Iran Seizes Gujrati Ships passing through the Strait of Hormuz
█
```

Subscriber2- subscribes and receives broadcasted messages from topic Sports

```
Secure subscriber connected
Topic to subscribe: Sports

Received: SUBSCRIBED:Sports

Received: MESSAGE:Sports:RR won against LSG by 40 runs

Received: MESSAGE:Sports:next match
█
```

GitHub Link:

https://github.com/idwmu/CN_2_E